Class-Incremental Semantic Segmentation of Aerial Images via Pixel-Level Feature Generation and Task-Wise Distillation

Lianlei Shan, *Graduate Student Member, IEEE*, Weiqiang Wang, *Member, IEEE*, Ke Lv, *Senior Member, IEEE*, and Bin Luo, *Senior Member, IEEE*

***Abstract*— Deep neural networks achieve significant progress in semantic segmentation but still suffer from the catastrophic forgetting problem, i.e., networks will forget old classes as they learn new ones. In this article, we propose an effective class-incremental segmentation framework without storing old data. Specifically, to alleviate the issue of catastrophic forgetting, we present two important modules, i.e., the pixel-level feature generation (PFG) module and the task-wise knowledge distillation (TKD) module. The PFG module is designed to constantly generate any number of features of the old classes to keep the old memory. The PFG module is the first attempt to use the generative method in class-incremental segmentation of aerial images, and it abandons the previous image generation approach but generates pixel-level features, which is more suitable for the segmentation task. Meanwhile, the proposed TKD module is specially designed for class incremental tasks, and it only compares classes in the same learning step (task), thus avoiding the squeezing of new classes into old classes when the output is normalized (softmax), making distillation more effective. Sufficient experiments show that our method is remarkably effective and achieves more than 4.5% gains compared with state-of-the-art methods and more than 13% compared with baselines on all learning conditions. The ablation studies show that the PFG module and the TKD module are both indispensable. Besides, the proposed framework can be well combined with any existing class incremental learning method to achieve better performance.**

***Index Terms*— Class-incremental learning, pixel-level feature generation (PFG), semantic segmentation, task-wise knowledge distillation (TKD).**

I. Introduction

SEMANTIC segmentation is a fundamental task in computer vision, which assigns a label to each image pixel. Recently, semantic segmentation technology has developed by leaps and bounds with the emergence of deep neural networks [1], [2], [3] and the availability of large-scale human-annotated datasets [4], [5]. Among them, for the semantic segmentation task of aerial images, there have been a lot of excellent works to optimize the model from the aspects of increasing receptive field [6], [7], spatial relation [8], valuable context [9], and adversarial approaches [10].

However, under the traditional setting, all training samples are always accessible, and all classes are learned at once [1], [11]. This setup is quite limited for real-world applications. When we apply the previous full-class segmentation networks to aerial images, we have to completely collect all the classes at once; otherwise, the model must be retrained from scratch when a new class is added. Since different environments may have different geographic classes, it is difficult to collect all classes at a time. On the one hand, training a new model means that a lot of computational resources used to train the old model are almost wasted, and the only role of the old model is to initialize the parameters of the new model. On the other hand, due to data security and other reasons, such as data loss, sometimes, it cannot be guaranteed that the data to train the old model can be always obtained. In a more realistic scenario, the segmentation model should be able to continually learn new classes without retraining from scratch, which is called class-incremental semantic segmentation (CSS). The CSS first appears in medical applications [12] and recently has been extended to the aerial images' segmentation [13]. Deep learning approaches to CSS face the challenge of *catastrophic forgetting* [14]. Catastrophic forgetting means that, when neural networks are learning new classes (tasks), they often hugely forget the previously learned knowledge [15]. Catastrophic forgetting is a real challenge for class-incremental learning applications based on deep learning, especially when previously viewed data are not allowed to be stored for privacy reasons.

Previous methods of CSS utilize distillation [16], [17], [18], producing pseudolabels [17], [19] to preserve the memory of old classes. However, using only these methods cannot achieve a satisfying balance between learning new classes (plasticity) and keeping the memory of old classes (stability). Aerial images are generally class imbalanced [4], which can cause a vicious circle for class incremental segmentation. For example, if a class cannot achieve good performance in its learning step, it is less likely to appear in subsequent knowledge

Manuscript received 18 July 2022; revised 11 September 2022 and 27 October 2022; accepted 21 November 2022. Date of publication 21 December 2022; date of current version 30 December 2022. This work was supported in part by the NSFC Key Projects of International (Regional) Cooperation and Exchanges under Grant 61860206004 and in part by the NSFC Projects under Grant 61976201. *(Corresponding author: Lianlei Shan.)*

Lianlei Shan and Weiqiang Wang are with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: shanlianlei18@mails.ucas.edu.cn; wqwang@ucas.ac.cn).

Ke Lv is with the School of Engineering Science, University of Chinese Academy of Sciences, Beijing 100049, China.

Bin Luo is with the MOE Key Laboratory of Signal Processing and Intelligent Computing, School of Computer Science and Technology, Anhui University, Hefei 230601, China.

Digital Object Identifier 10.1109/TGRS.2022.3231351

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

distillation or pseudolabels, making forgetting the class more intense. Simply put, it is more difficult to keep the memory of classes with poor accuracy. It is a really severe problem and seriously affects the performance of class-incremental segmentation in aerial images but has been ignored by previous general distillation- or pseudolabel generation-based methods. We found this problem and solved it from the aspect of the old data generation and the improvement of distillation.

The proposed pixel-level feature generation (PFG) module can generate any number of features for each class to retain the memory of old classes. Intuitively, the abovementioned problem can be easily solved by generating more features for the classes with poor performance. Meanwhile, the previous general distillation methods are discarded, and a distillation especially for incremental segmentation, called task-wise knowledge distillation (TKD), is proposed, making classes in the same learning task compete with each other, while the classes of different learning tasks do not interfere with each other. In this way, when distilling, the logits of the old classes are not squeezed by the new classes, making distillation more effective. The PFG module and the TKD module will be introduced, respectively, in the following parts.

A. Pixel-Level Feature Generation Module

Some methods in class-incremental classification tasks keep the balance between old and new classes by retaining the most discriminating features [20], [21] or generating images of old classes [22], [23] and achieve impressive results. We follow the research line of the generation approach. However, different from classification tasks, segmentation is a dense prediction task that requires pixel-level classification. Therefore, the image generator previously used in the classification tasks cannot be well employed because pixel-level labels cannot be obtained accurately, and the pixel-level label is necessary for the segmentation task. To solve the problem, inspired by the zero-shot segmentation [24], [25], we first design a complete framework of PFG for CSS.

Pixel-level features are much more challenging to generate than image-level features. Therefore, we introduce word embedding produced by word to vector (W2V) [26] with definite classes as input for the generator rather than one-hot labels. Besides, we use variational autoencoders (VAEs) [27] instead of the traditional conditional GAN [28] as the generator, which increases the stability of the generation. Moreover, the generated features from VAE are combined with extracted true features and then as the input to the classifier to enhance training. Multiple losses are designed to ensure the consistency and diversity of generated features. Through the above operations, the proposed PFG module can produce valuable features to keep the memory of old classes.

B. Task-Wise Knowledge Distillation Module

The distillation methods [20], [29], [30] used before in CSS also involve imbalance. General knowledge distillation (GKD) is commonly used in the previous CSS networks [29], [30], where softmax is applied to the output of all learned classes. Other works [20] calculate the sigmoid of each class and then calculate distillation, which we call class-wise knowledge distillation (CKD). GKD puts all classes in the same comparison, so the output of the new classes will squeeze the old classes, making the logits of the old classes extremely small. This phenomenon makes distillation much less effective, thus leading to the decline of the memory of old classes. CKD only considers itself without any comparison to other classes, resulting in poor results. We abandon the previous distillation methods and introduce a distillation where its softmax is calculated with classes in the same learning step to CSS, which is called TKD [31]. TKD is to compare the classes in the same learning step (task), and the classes of different learning steps do not interfere with each other. In this way, the new classes will not squeeze the old classes, and the comparison between different classes is allowed. These characters of TKD perfectly fit the requirements of class incremental learning.

In a nutshell, through the PFG module and the TKD module, catastrophic forgetting can be well alleviated. Our main contributions can be summarized as follows.

1) We propose an effective framework for CSS, which can generate high-quality pixel-level features and can perform distillation effectively through learning steps. Besides, the proposed framework can easily solve the vicious cycle caused by the imbalance of the original data for incremental segmentation.
2) The generative-based method is first introduced to CSS for aerial images. The previous image generation methods are abandoned, and a PFG module is proposed, which can generate pixel-level features with explicit classes and is very suitable for segmentation tasks. The PFG module can generate any amount of features of different classes; therefore, it is easy to guarantee balance between new and old classes, as well as between different old classes.
3) A distillation method specifically for class incremental segmentation, called TKD, is proposed to ensure that classes can be compared with each other, and new classes do not squeeze old ones.

II. Related Works

A. Class-Incremental Learning

Catastrophic forgetting [14], [15], [32] has been extensively studied in the classification task, and current technologies can be divided into the following categories [33], [34]: regularization methods [35], parameter isolation methods [36], [37], dynamic architecture methods [38], and replay-based methods [22], [23]. Regularization-based methods are the most widely used methods, which can be divided into two subcategories: the penalty calculation method and the knowledge distillation method. The penalty calculation methods [39], [40] protect important weights within the model to prevent forgetting. In knowledge distillation methods [22], [31], the teacher (old) model transfers or memorizes knowledge related to previous tasks to the student (current) model, and the student model is also trained to learn additional tasks. Parameter isolation methods reserve a subset of weights for a particular task to avoid degradation. Dynamic architecture methods generate

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

new branches for new tasks. Replay-based models utilize examples stored [20], [21] or generated [22], [23] during learning new tasks.

The previous methods in CSS of aerial images are all based on knowledge distillation or penalty, and our work is the first one to use a generative approach.

B. Class-Incremental Semantic Segmentation

Deep learning has made tremendous achievements in segmentation [9], [41]. However, these methods require data can be accessed all the time during training. If the data for a new task (for example, a new class) are only continually available, they will suffer from catastrophic forgetting. Early works focused on the problem in medical imaging [12], [42]. Recently, CSS of aerial images has been proposed, including datasets and corresponding methods [43]. In [30] and [44], the distillation losses at the output and intermediate feature levels are designed to alleviate forgetting, and the parameter of the encoder is also frozen. Cermelli et al. [29] found the distribution shift of background when merging previous and current classes. This background shift is solved by the unbiased version of cross-entropy and the output level knowledge distillation loss. Yu et al. [45] employed self-training for CSS. Li et al. [18] proposed a twin-auxiliary architecture that adds an auxiliary binary classification task. Rong et al. [17] proposed two novel historical information-guided modules: the feature global perception module and the label reconstruction (LR) module, which can enable the current model to pay more attention to the region related to the old categories and retrieve pixels belonging to the learned classes from the background to handle the background shift problem. Shan et al. [19] proposed a distillation from not only the final feature level but also the intermediate feature level to ensure the memory of the old classes.

It can be seen that all of the previous methods use distillation to preserve the memory of the old classes. The problem with this strategy is that there are no additional measures to remedy the classes with poor performances, i.e., for the performance of the old classes, the new model will only be worse than the old model. For those classes with small sample sizes and poor performances, forgetting is more rapid and irreversible. In order to solve this problem, we, for the first time, propose a generative method in class-incremental segmentation, which can selectively generate more features for the classes with poor performance, so as to easily solve the problem of imbalance. On the other hand, we have improved the distillation method at the output level to ensure both the comparison of different classes and the no-squeezing between the old classes and new ones.

C. Learning Without Forgetting via Word Vectors

Word vectors have shown promising success on various computer vision tasks, such as image/video captioning and visual question answering [46]. More recently, some studies [47] also show that word vectors can also help to learn without forgetting. Rahman et al. [47] used semantic word vectors in any-shot object detection to simultaneously detect unseen targets and fewer shooting targets. Word vectors help reduce the forgetting of the classes during the fine-tuning (FT) process. Cheraghian et al. [48] used word vectors in their proposed pipeline to solve the few-shot class incremental learning to reduce catastrophic forgetting. They used a distillation method and semantic word vectors to solve the forgetting problem during the training phase. Zhu et al. [49] used the word vector to solve the problem of object detection with few shots, which introduced a way to learn new objects from visual information and semantic relationships. Cheraghian et al. [50] used visual and semantic class information to form class prototypes on mixed subspaces defined on base class instances.

In conclusion, word vectors have been demonstrated to be valuable in feature generation, and we also combine this advanced technology with our framework.

D. Generative Model for Synthesized Feature

Generating features to improve the performance of classification networks has been an interesting and effective approach in recent years. In challenging scenarios with limited or no available data, generating artificial features helps the model deal with extreme imbalances in training data. For example, Xian et al. [51] utilized a generative adversarial network (GAN) to synthesize features using class-level semantic information. They used these features in the zero-shot learning setup and confirmed that the generated features contained sufficient discriminative properties. Schonfeld et al. [52] used a VAE for the same purpose. Similarly, Xian et al. [53] processed arbitrary shot learning settings, i.e., zero-shot and few-shot, in a unified feature generation framework that operated in both inductive and transductive learning settings. Liu et al. [54] proposed a generative image-level feature replay approach. Gu et al. [24] designed a context-aware module to make the generated features contain context information.

We follow the idea of the abovementioned approaches and utilize semantic embedding to generate visual features, which is easier for training and can be more diverse.

III. Method

In this section, we introduce the proposed approach in detail. In Section III-A, we introduce the definition and setups of CSS. In Section III-B, we give an overview of the framework, including the structure of the framework, the overall procedure, and the iterative training process of the segmentation network and the PFG module. In Section III-C, the PFG module is introduced in detail, including how to ensure the consistency and diversity of generated features and the incremental learning of the PFG module. TKD is introduced in Section III-D.

A. Definition and Setups of Class-Incremental Semantic Segmentation

We denote input images with $\mathbf{I}$, the set of classes with $\mathcal{C} = \{c_i\}_{i=0}^{M}$, and output space with $\mathbf{Y}$ (i.e., the segmentation map). $\mathbf{I} \in \mathbb{R}^{H\times W\times 3}$, and $\mathbf{Y} \in \mathbb{R}^{H\times W\times C}$. H and W represent the height and width of images. In the standard supervised setting,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATION OF MAIN SYMBOLS USED IN THIS ARTICLE

Description	One-hot Label	Word Embedding	Latent Code
Old Classes	$\tilde{\mathbf{Y}}$	$\tilde{\mathbf{W}}$	$\tilde{\mathbf{Z}}$
Current Classes	$\mathbf{Y}$	$\hat{\mathbf{W}}$	$\hat{\mathbf{Z}}$

it is assumed that all data are available at once, and all classes are learned at one training step.

In the class-incremental learning setting, the training is achieved through multiple steps (tasks), and each step learns one or more novel classes. Only a subset of the training data is accessible at one training step. Formally, given $T+1$ tasks, we split $\mathcal{C}$ into $T+1$ subsets $\mathcal{C}_0, \mathcal{C}_1, \ldots, \mathcal{C}_T$, and $\mathcal{S}_t = \mathcal{C}_0 \cup \mathcal{C}_1, \ldots, \cup \mathcal{C}_t$. If $i \neq j$, then $\mathcal{C}_i \cap \mathcal{C}_j = \emptyset$. Given the training set, $\mathcal{T} = \{(\mathbf{I}_t, \mathbf{Y}_t)\}_{t=0}^{T}$. Task t introduces new classes $\mathcal{C}_t$ and only accesses dataset $\mathcal{T}_t = \{(\mathbf{I}_t, \mathbf{Y}_t)\}$, i.e., the network can only obtain the data and labels of classes learned at the current step. The goal of task t is to train a model that can segment the objects belonging to class $\mathcal{S}_t$, i.e., when the network learns the new classes, it also maintains the memory of previously learned old classes. At learning step t, the previous label set $\mathcal{S}_{t-1}$ is expanded by a set of novel classes $\mathcal{C}_t$ to form a new label set, i.e., $\mathcal{S}_t = \mathcal{C}_t \cup \mathcal{S}_{t-1}$.

B. Overview of the Framework

In this section, we introduce an overview of the framework. First, we introduce the symbol definition and the overall structure of the framework, i.e., the framework is made up of which modules. Next, we introduce the overall process, i.e., how the input becomes the output. Finally, we introduce the training procedure of the framework.

1) Symbol Definition: Before introducing the procedure in detail, we first make a definition of symbols. The generated pseudolabel $\tilde{\mathbf{Y}}_t$ in Table I contains the previously learned classes $\mathcal{S}_{t-1}$, while $\mathbf{Y}_t$ contains the classes $\mathcal{C}_t$ learned in the current step. $\tilde{\mathbf{Y}}_t$ and $\mathbf{Y}_t$ are both the one-hot labels. $\tilde{\mathbf{W}}$ and $\hat{\mathbf{W}}$ are the output of W2V [26]. $\tilde{\mathbf{Z}}$ is random Gaussian noise, and $\hat{\mathbf{Z}}$ is the latent code from *halfVAE*. Y_t and $\tilde{Y}_t$ can be single-class or multiclass labels. If there is only one learned (old) class at step t, then $\tilde{Y}_t$ is one class. If there is more than one old class, then $\tilde{Y}_t$ is multiclass. Y is set according to the current learning classes. If learning one class at step t, i.e., 1-1 in the experimental setting, then Y_t is a single class. If learning more than one class at step t (2-2 or 3-3) and the image contains more than one current learning new class, then Y_t is multiclass.

Table I clearly shows the meanings of each symbol, which are important for understanding the whole process. The subscript t indicates that the current is the tth learning step, and no subscript t means that the operation is the same for different steps.

2) Architecture Introduction: The overall framework architecture is shown in Fig. 1. Since the architecture is the same in each step, we omit the subscript t in introducing the architectures. The network mainly consists of two parts: segmentation networks (including feature extractor and classifier in Fig. 1) and the PFG module (including feature generator and discriminator in Fig. 1). The TKD module is the distillation calculation of the output part. The segmentation network and the PFG module are introduced in the following.

The segmentation network can be any existing segmentation network. We use Deeplabv3 [55] as our basic segmentation network, which shows excellent performance in semantic segmentation. Any segmentation network can be divided into two parts: backbone E for feature extraction and classifier C (for example, one or two 1×1 convolutional layers [55]). In normal full-class training, given an input image $\mathbf{I}$, the backbone outputs its real features $\mathbf{X}$ and passes them to the classifier C to obtain the segmentation result. The only difference with the original networks is that, after feature extraction, a module is added, and we call it *halfVAE*. The role of *halfVAE* is to ensure the generated features consistent with real features extracted from the images, which will be introduced next.

The PFG module is composed of the W2V, the generator G, and the discriminator D. W2V changes the one-hot class labels ($\tilde{\mathbf{Y}}$ or $\mathbf{Y}$) to semantic word embedding ($\tilde{\mathbf{W}}$ or $\hat{\mathbf{W}}$). The generator G is to transform the combination of the semantic word embedding and latent code ($\tilde{\mathbf{Z}}$ or $\hat{\mathbf{Z}}$) into features ($\tilde{\mathbf{X}}$ and $\hat{\mathbf{X}}$) with explicit classes. The classes of the generated feature come from the one-hot labels that are the input. The discriminator is designed to make the generator training better, so as to make the generator produce more realistic and diverse features. *HalfVAE* can also be considered as part of the PFG module because its output is also taken as the input of the generator G. In the following introduction, we put *halfVAE* into the PFG module to introduce.

Note that only the segmentation network part is used when the network comes to testing, and others are all discarded. Next, we introduce the overall procedure, i.e., what operations the input goes through to become the output.

3) Overall Procedure: The overall procedure is shown in Fig. 1. Now, we introduce the procedure. We use the learning step t as a representative to introduce the overall process. In the learning step t, W2V takes pseudolabel $\tilde{\mathbf{Y}}_t$ (or the true label $\mathbf{Y}_t$) to generate semantic word embedding maps $\tilde{\mathbf{W}}_t$ (or $\hat{\mathbf{W}}_t$), and then, $\tilde{\mathbf{W}}_t$ (or $\hat{\mathbf{W}}_t$) is concatenated with the latent code map $\tilde{\mathbf{Z}}_t$ (or $\hat{\mathbf{Z}}_t$) on the channel dimension. $\tilde{\mathbf{Z}}_t$ is generated by random noise, and $\hat{\mathbf{Z}}_t$ is the output of $halfVAE_t$. Generator G_t takes concatenated latent code and word embedding as input and outputs fake features $\tilde{\mathbf{X}}_t$ (for old classes) and $\hat{\mathbf{X}}_t$ (for current classes). Then, discriminator D_t is used to distinguish real (extracted) or fake (generated) features, and classifier C_t is used to output the segmentation results. Note that only the features for current learning classes are involved in the discriminator, i.e., only $\tilde{X}_t$ (generated features of current learning classes) and X_t (extracted real features of current learning classes), for the calculation for discrimination loss.

Although the procedure of each learning step is the same, the training for the first and subsequent steps is different. We introduce the learning process for the first and subsequent steps in the following part.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

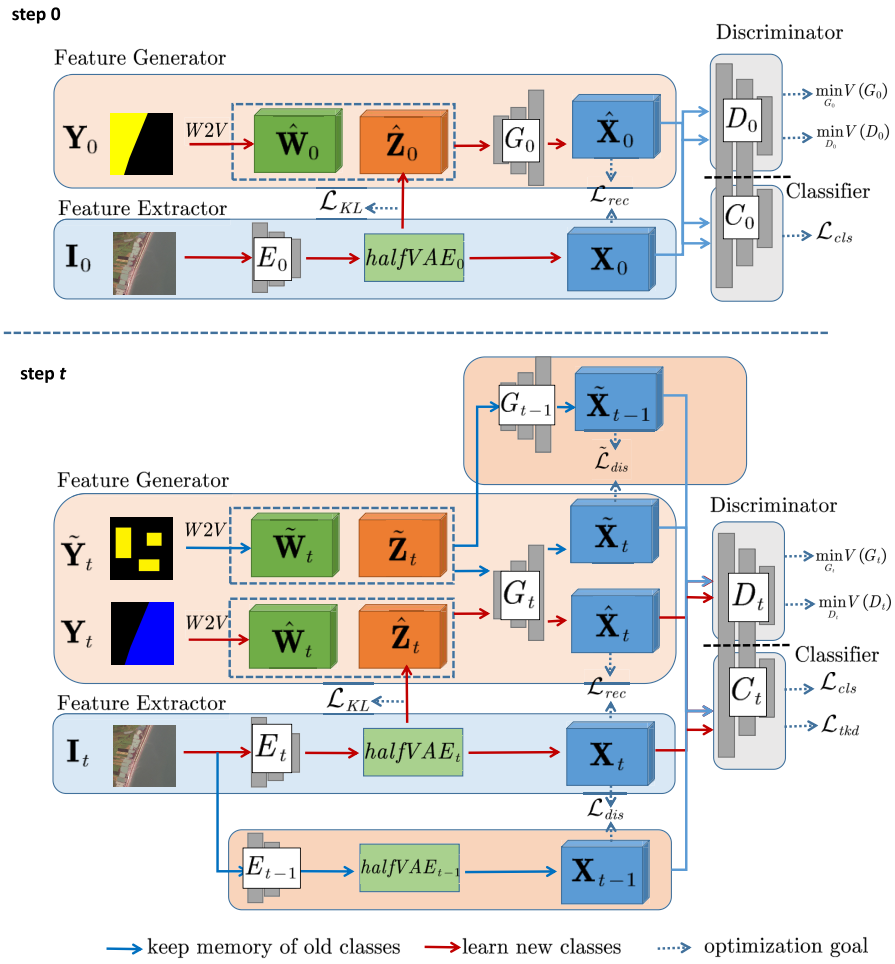


Fig. 1. Overall structure of the network at training step 0 and step t ($t \geq 1$). Learning class agriculture (yellow) at step 0 and the water (blue) at step t. The network mainly consists of two parts: segmentation networks (including feature extractor E_t, $halfVAE_t$, and classifier C_t) and the PFG module (including feature generator W2V, G_t, and discriminator D_t). TKD is used to generate loss $\mathcal{L}_{\text{tkd}}$.

4) Training Procedure: The training procedure includes class-incremental learning of the segmentation module and the PFG module, and the detail is shown in Algorithm 1. At step 0, the PFG module and the segmentation network are trained together. The overall process is shown in Fig. 1.

At step t ($t \geq 1$), the segmentation network is trained first, and then, the well-trained segmentation network is used to train the PFG module. The PFG module then generates pixel-level features of old classes when the segmentation is trained to learn new classes so that the segmentation network can keep the memory of the old classes. In general, the segmentation network is trained with the help of the PFG module, which generates pixel-level features of old classes to keep the memory of old classes. Then, the well-trained segmentation network helps the PFG module learn generating features of new classes. The iteration keeps repeating. Note that the model at step t is initialized by the model parameters at step $t-1$.

Besides, the previously used feature distillation loss $\mathcal{L}_{\text{dis}}$ and classification $\mathcal{L}_{\text{cls}}$ are still kept as in [30], where $\mathcal{L}_{\text{dis}}$ is the mean square error (mse) loss between features and $\mathcal{L}_{\text{cls}}$ is the cross-entropy loss. That is, we use one of the previous class-incremental learning methods, ILT, as our baseline. The output level distillation $\mathcal{L}_{\text{tkd}}$ is introduced in the following. During the test, the network only contains the semantic segmentation part (feature extractor and classifier).

It can be seen that the network retains the memory of the old classes by generating the features of the old classes. Therefore, how to generate the features accurately and ensure diversity is the key to competitive performance. We will introduce the specific process of the PFG module in Section III-C. The calculation method of TKD and its advantages compared with GKD and CKD will be introduced in Section III-D.

C. Pixel-Level Feature Generation Module

The PFG module is used to generate pixel-level features, which include W2V, generator G, *halfVAE* (whose output is the input to the generator), and discriminator D, as shown in Fig. 1. One-hot labels become semantic word embedding through W2V, and then, the semantic word embedding and the latent code are concatenated on the channel dimension. The concatenated result is as input to generator G to get the generated features. Discriminator D is used to discriminate

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Learning Procedure

Input : Training data sequence $\mathcal{T}_0, \ldots, \mathcal{T}_T$
Output: The segmentation network of the last step T: $\mathcal{M}_T = \{E_T, halfVAE_T, C_T\}$.
The PFG module of the last step T: $\mathcal{P}_T = \{G_T, D_T\}$.

```
for t = 0, ..., T do
    if t = 0 then
        Process: Train the segmentation network M_0 and through data T_0. Train the PFG module P_0 through X_0 and data T_0, where X_0 = halfVAE_0(E_0(I_0)).
    end
    else
        Process 1: Train the segmentation network M_t through data T_t, X̃_{t-1} and X_{t-1}. X̃_{t-1} = G_{t-1}(W̃_t, Z̃_t) and X_{t-1} = halfVAE_{t-1}(E_{t-1}(I_t)). I_t is the images of T_t.
        Process 2: Train the PFG module P_t through X_t and X̃_{t-1}, where X̃_{t-1} = G_{t-1}(W̃_t, Z̃_t), and X_t = halfVAE_t(E_t(I_t)).
    end
end
```

whether features are true or false so that generators can be trained well.

We use W2V instead of putting the one-hot labels directly into the generator because the strategy with W2V works better and W2V is easy to implement [26]. The input can be the label of the current step; thus, the PFG module generates the features of current learning classes. The input can also be a randomly generated label of old classes so that the features of the old class can be generated. We can determine the proportion of features of each old class via the performance of old classes [such as Intersection over Union (IoU)]. In the implementation, we invert the IoU of each class to determine the proportion of the class so that the PFG module can generate more features of the classes with poor performance, which is an intuitive but very effective method. This can easily solve the problem of class imbalance.

The key to feature generation is how to make the generated features and extracted true features as consistent as possible (consistency) and ensure the feature diversity to avoid mode collapse [56] (diversity). The PFG module is also class-incremental learning, so the PFG module also needs to learn new classes and maintain the memory of old classes. We will introduce the two aspects in the following parts.

1) Way to Keep Diversity and Consistency: halfVAE is used to ensure that the generated features are consistent with the real features, and the detailed structure of *halfVAE* is shown in Fig. 2. The front part (the process of obtaining $\mathbf{Z}_t$ from $\mathbf{F}_t$) is consistent with VAE, and operations at the latter part are to ensure that the extracted features also meet certain distributions. *halfVAE* takes $\mathbf{F}_t$ from the encoder (backbone) as input and outputs real feature $\mathbf{X}_t$ and corresponding contextual latent code $\mathbf{Z}_t$. First, the feature $\mathbf{F}_t$ extracted by encoder

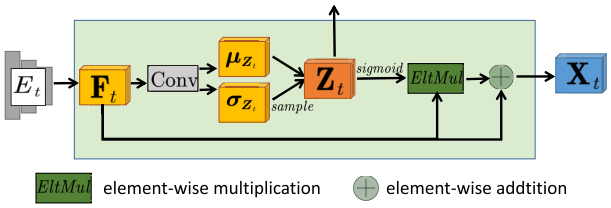


Fig. 2. Overview of the *halfVAE* module. Conv represents the convolutions, EltMul denotes the element-wise multiplication, and "+" denotes the element-wise addition. The input of *halfVAE* is extracted features from the encoder. There are two outputs: one is for the classifier (the right direction) and the other is sent to the generator (the upward direction). The color is consistent with Fig. 1.

obtains $\boldsymbol{\mu}_{\mathbf{Z}_t}$ and $\boldsymbol{\sigma}_{\mathbf{Z}_t}$ through convolutions. Then, the contextual latent code $\mathbf{z}_{t,i}$ for the ith pixel can be sampled from Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}_{\mathbf{Z}_{t,i}}, \boldsymbol{\sigma}_{\mathbf{Z}_{t,i}})$ by using $\mathbf{Z}_{t,i} = \boldsymbol{\mu}_{\mathbf{Z}_{t,i}} + \epsilon\boldsymbol{\sigma}_{\mathbf{Z}_{t,i}}$, and ϵ is a random scalar sampled from $\mathcal{N}(0, 1)$. To enable stochastic sampling during inference, we employ a Kullback–Leibler (KL)-divergence loss to enforce $\mathcal{N}(\boldsymbol{\mu}_{\mathbf{Z}_{t,i}}, \boldsymbol{\sigma}_{\mathbf{Z}_{t,i}})$ to be close to standard Gaussian distribution $\mathcal{N}(\mathbf{0}, \mathbf{1})$, i.e.,

$$\mathcal{L}_{\mathrm{KL}} = \mathcal{D}_{\mathrm{KL}}\big[\mathcal{N}(\mathbf{0}, \mathbf{1}) \| \mathcal{N}\big(\boldsymbol{\mu}_{\mathbf{Z}_{t,i}}, \boldsymbol{\sigma}_{\mathbf{Z}_{t,i}}\big)\big]. \tag{1}$$

Meanwhile, inspired by zero-shot semantic segmentation [24], [25], we element-wise multiple the $\mathbf{Z}_t$ after applying sigmoid activation with feature $\mathbf{F}_t$ as residual addition, i.e.,

$$\mathbf{X}_t = \mathbf{F}_t + \mathbf{F}_t \odot \phi(\mathbf{Z}_t) \tag{2}$$

where $\odot$ represents the element-wise multiplication and ϕ denotes the sigmoid. In this way, the extracted true features can be consistent with the generated features to the greatest extent.

a) Diversity: We build the one-to-one correspondence (bijection) between inputted pixel-wise contextual latent code and outputted pixel-wise feature. It has been shown in [57] that the bijection between input latent code and output could mitigate the mode collapse problem, so our model can generate more diverse features from semantic word embedding. We enforce the latent contextual code to follow standard Gaussian distribution to get various contextual latent codes.

b) Consistency: In order to ensure the generated features consistent with the extracted features, we employ two optimization goals, mse loss, and adversarial goal. Their locations can be found in Fig. 1. $\mathcal{L}_{\mathrm{rec}}$ is the mse loss between generated feature and extracted feature, shown as

$$\mathcal{L}_{\mathrm{rec}} = \sum_i \big\|\hat{\mathbf{X}}_{t,i} - \mathbf{X}_{t,i}\big\|_2^2. \tag{3}$$

Following [24], the adversarial goal is

$$\begin{aligned}\min_{D_t} V(D_t) = \sum_i (&\mathbb{E}_{\mathbf{X}_{t,i}\sim p_{\mathrm{data}}}\big[(D_t(\mathbf{X}_{t,i}) - 1)^2\big] \\ &+ \mathbb{E}_{\mathbf{Z}_{t,i}\sim p_z}\big[(D_t(G_t(\mathbf{Z}_{t,i})))^2\big]) \\ \min_{G_t} V(G_t) = \sum_i (&\mathbb{E}_{\mathbf{Z}_{t,i}\sim p_z}\big[(D_t(G_t(\mathbf{Z}_{t,i})) - 1)^2\big])\end{aligned} \tag{4}$$

where the discrimination score from D_t is normalized within [0, 1] by a sigmoid function, with target 1(resp., 0) indicating real (resp., fake) pixel-wise features. Both $\mathcal{L}_{\mathrm{rec}}$ and the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

adversarial goal is designed to make the generated features as consistent as possible with the real features.

In addition to these two direct optimization goals to guarantee consistency, classification loss $\mathcal{L}_{\text{cls}}$ can also guarantee the consistency of the two features. Intuitively, if the generated feature is different from the real extracted feature, the classification result will produce errors, so the network will learn to generate features more realistically.

2) Class-Incremental Learning of the PFG Module: Fig. 1 shows the steps of class-incremental learning from the view of the segmentation network. In fact, the PFG module is also incrementally trained. We introduce the class-incremental learning of the PFG module from two aspects: keeping the memory of old classes and learning new classes.

a) Keeping memory of old classes: When the module takes $\tilde{\mathbf{Y}}_t$ as input, the output of generators learned at step $t-1$ and step t will compute $\tilde{\mathcal{L}}_{\text{dis}}$ as

$$\tilde{\mathcal{L}}_{\text{dis}} = \sum_i \left\| \tilde{\mathbf{X}}_{t,i} - \tilde{\mathbf{X}}_{t-1,i} \right\|_2^2 \tag{5}$$

where $\tilde{\mathbf{X}}_t = G_t\,(\tilde{\mathbf{W}}_t, \tilde{\mathbf{Z}}_t)$ and $\tilde{\mathbf{X}}_{t-1} = G_{t-1}\,(\tilde{\mathbf{W}}_t, \tilde{\mathbf{Z}}_t)$. The loss function keeps the output of the old and new PFG modules as consistent as possible to keep the module's memory of the old classes.

b) Learning new classes: Learning new classes is similar to keeping the memory of old classes, except that the input changes from the pseudolabel $\tilde{\mathbf{Y}}_t$ of the old classes to the true label $\mathbf{Y}_t$ of the current learning classes. When the input is the label of the current learning classes, generator G is learning new classes. By calculating $\mathcal{L}_{\text{rec}}$ between extracted features $\mathbf{X}_t$ and generated features $\hat{\mathbf{X}}_t$, as shown in (3), the generated features of the new class will be consistent with the extracted features. Meanwhile, $\mathcal{L}_{\text{cls}}$ will make the features generated by the generator approximate the real features, so as to make the classification correct.

Through the above two measures, the PFG module can also carry out class-incremental learning. Generally, the segmentation network and the PFG module can help each other to both achieve class-incremental learning. It is the dissynchronization of learning between the two that enables each learning to obtain the help of the other, i.e., the PFG module can help the segmentation network to keep the memory of old classes, while the segmentation network can help the PFG module better generate the features of new classes.

D. Task-Wise Knowledge Distillation Module

The TKD module is used to calculate the distillation loss of the old model $\mathcal{M}_{t-1}$ and the new model $\mathcal{M}_t$ at the output level. In this section, we introduce the advantages and specific implementation of the TKD module.

GKD [29], [30] and CKD [20] are usually used in class-incremental segmentation. Their difference compared with TKD is shown in Fig. 3. They differ in the normalization function (sigmoid or softmax) for the network output. GKD is to normalize (softmax) all the previously learned classes together and then calculate KL divergence. CKD is to calculate the sigmoid for each class and then calculate KL divergences.

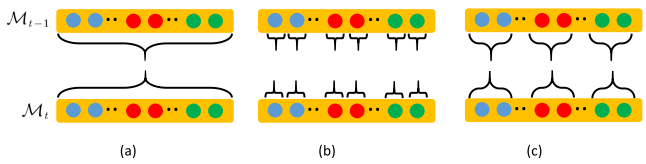


Fig. 3. Comparison of different distillation methods. A dot represents the network's output to a class, and the same color represents the classes trained at the same step. Brackets represent normalization ranges, and normalization ways include softmax and sigmoid. (a) GKD. (b) CKD. (c) TKD.

TKD is to calculate softmax for classes learned at the same step, and classes of different learning steps do not interfere with each other.

Advantages of TKD for the CSS Task: GKD mixes currently learning classes with all previously learned old classes for the calculation of softmax, which will lead to results biased toward the current new classes [58]. In the CSS task, while learning the current class, the accessed images can contain pixels belonging to the old classes though the pixel's label is set to the background class. That is, in the segmentation task, an image contains multiple classes and inevitably can contain pixels of the old classes. This situation is also covered in detail in DisALL [19]. Through our experiments, GKD usually performs better than TKD in the classification task because the advantage of TKD can only be fully shown when the data containing old classes is the input. When data that only contain current classes are the input, the character of GKD that output is biased toward the new class is conducive to learning the new class. Therefore, GKD is not suitable for CSS tasks. CKD is individually calculated for each class via the sigmoid function, so it is not compared with other classes, which results in poor differentiation.

Formally, we use $\mathcal{M}_t(\mathbf{I}_i)$ to denote the output of model $\mathcal{M}_t$ for the ith pixel of image $\mathbf{I}$ before normalization. $\boldsymbol{p}_k^\tau$ denotes the probability vector of classes learned at step k with temperature scaling parameter τ. Then, the loss for TKD defined for an input image $\mathbf{I}$ at training step t is given as follows:

$$\mathcal{L}_{\text{tkd}}(\mathbf{I}) \triangleq \sum_i \sum_{k=0}^{t-1} \mathcal{D}_{\text{KL}}\left(\boldsymbol{p}_k^\tau(\mathcal{M}_{t-1}(\mathbf{I}_i)) \,\|\, \boldsymbol{p}_k^\tau(\mathcal{M}_t(\mathbf{I}_i))\right) \tag{6}$$

where $\mathcal{D}_{\text{KL}}(\cdot\|\cdot)$ is the KL divergence. $\mathcal{M}_t$ is the model learned at task t, and $\mathcal{M}_{t-1}$ is the model learned up to task $t-1$. We use $\mathcal{M}_t^{(c)}(\mathbf{I}_i)$ to denote the network output to class c of $\mathbf{I}_i$. $p_{k,c}^\tau$ denotes the probability for classes c learned at step k ($c \in \mathcal{C}_k$), and its calculation is

$$p_{k,c}^\tau(\mathcal{M}_t(\mathbf{I}_i)) = \frac{e^{\mathcal{M}_t^{(c)}(\mathbf{I}_i)/\tau}}{\sum_{s\in\mathcal{C}_k} e^{\mathcal{M}_t^{(s)}(\mathbf{I}_i)/\tau}}. \tag{7}$$

In simple terms, TKD is to normalize the classes within the same learning step and then the calculation of KL divergence (i.e., knowledge distillation).

IV. Experiments

We do sufficient experiments and also do complete comparisons with state-of-the-art (SOTA) methods. First, the details of experiments are introduced in Section IV-A, including

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

the used dataset, the preparation of training data and testing data, evaluation metrics, hyperparameters, and the introduction of comparison methods. In Section IV-B, we introduce and analyze the results under different settings and under different segmentation networks, limitations and possible solutions, and computation cost analysis. Next, we introduce the results on Potsdam, Vaihingen, and iSAID. The results combined with other class-incremental learning methods are introduced in Section IV-D. Ablation studies are introduced in Section IV-E, including the role of each module and the comparison with others. In Section IV-F, we explore how the different proportions of the old and new classes affect the final results. The results with different weights of the losses are shown in Section IV-G. The influence of different learning orders on the final results is introduced in Section IV-H.

A. Implementation Details

1) Dataset: We conduct experiments on the Deepglobe [4] dataset. Deepglobe consists of 803 images, each with a resolution of 2448 × 2448. In line with [59], we divide these images into training sets and test sets of 455 and 348 images, respectively. The dense pixel-level label is divided into six classes: urban, agricultural, rangeland, forest, water, and barren. The "unknown" region is not considered in the challenge as previous methods [19], [59].

2) Preparation of Training Data and Testing Data: We first divide the original large images into training and test sets, and then crop the images in patches, i.e., the training and test sets are split before the images are cropped. Then, the original images with the size of 2448 × 2448 are cropped to 512 × 512. In the cropping process, we first conduct sequential cropping and then random cropping to obtain the required number of patches, and there is no overlap when sequentially cropping. As a result, the training set contains 20 000 patches, and the testing set contains 10 000 patches. The dataset can cover all six classes. We adopt the same way to prepare the training data for each step of class-incremental learning as MiB did. Concretely, for step t, if class t is the target class learned at the current step, all the patches that contain pixels of class t in the training set are identified and collected to form the training data for step t. For each patch, all other pixels except pixels of the learned classes are labeled as background.

3) Evaluation Metric: We use mean IoU (mIoU) and mean F1-score (mF1) as the evaluation metrics. mIoU is the ratio of the intersection and union of two sets of true and predicted values, and can be evaluated by

$$\text{mIoU} = \frac{1}{K}\sum_{i=1}^{K}\frac{p_{ii}}{\sum_{j=1}^{K}p_{ij}+\sum_{j=1}^{K}p_{ji}-p_{ii}} \tag{8}$$

where K is the number of classes, i is the true class label, and j is the predicted label. p_{ij} denotes the number of pixels of class i predicted to class j.

mF1 score is defined as

$$\text{mF1} = \left(1+\beta^2\right)\frac{\text{precision}}{\beta^2\text{precision}+\text{recall}} \tag{9}$$

where β is the equivalent factor between precision and recall. β is set as 1. In addition, precision and recall are defined as

$$\text{precision} = \frac{\text{TP}}{\text{TP}+\text{FP}}$$
$$\text{recall} = \frac{\text{TP}}{\text{TP}+\text{FN}}. \tag{10}$$

4) Hyperparameters: We use a workstation with two NVIDIA GTX 3090 24G GPU cards, and the batch size is 12. We employ a polynomial decay rule with a power of 0.9 to train the network. For all methods, we use Deeplabv3 architecture [55] with ResNet-101 [60] as the backbone. The backbone is initialized by the model pretrained from ImageNet [61]. We follow [55] and train the network with SGD and the same learning rate strategy, momentum, and weight attenuation. Generator G is a multilayer perceptron (512 hidden neurons, Leaky ReLU, and dropout for each layer as [24] and [25]). Classifier C and discriminator D consist of two 1 × 1 convolutional layers and share the same weights in the first convolutional layer, which is the common setting in the semantic segmentation task. τ in output-level distillation is 1. The initial learning rate is 10^{-2} for the first learning step and 10^{-3} for the following steps as same with DisALL [19]. Since GPU memory requirements are an essential issue in semantic segmentation, in-place activation batch normalization is used, as proposed in [62]. The same data augmentation is applied as [55]. During training and testing, the images are cropped to 512 × 512. Each step of class-incremental learning involves 50 epochs of training. Other settings of hyperparameters are the same as those in DisALL [19]. The parameters of other networks for comparison are set as values given in the original papers. The W2V that we use is from [24] and [26], which can be obtained easily. Unless otherwise stated, the ratio of old and new classes is 1:1. We use cross-validation and select 20% from the training set as the validation set, which is consistent with DisALL [19] and MiB [29].

5) Loss: We use $\mathcal{L}_{\text{adv}}$ to represent the loss of adversarial generation, and the total loss function is

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{adv}} + \lambda_1\mathcal{L}_{\text{dis}} + \lambda_2\tilde{\mathcal{L}}_{\text{dis}} + \lambda_3\mathcal{L}_{\text{tkd}} + \lambda_4\mathcal{L}_{\text{rec}} + \lambda_5\mathcal{L}_{\text{kl}}. \tag{11}$$

The coefficient of each loss has been fully discussed in the previous class-incremental segmentation works, and we follow the previous works [19] and set $\lambda_1 = \lambda_2 = \lambda_3 = 100$, which obtains the best performance in [19]. In zero-shot semantic segmentation [24], there are also operations for feature generation, where the coefficient of $\mathcal{L}_{\text{rec}}$ is set to 100, and the coefficient of $\mathcal{L}_{\text{kl}}$ is set to 10, which also obtains the best results. We comply with the parameters of previous work and set λ_4 and λ_5 to 100 and 10. We also conduct experiments to verify the impact of these coefficients on the final results.

6) Introduction of Comparison Methods: To demonstrate the superiority of the proposed method, we compare it with some SOTA methods. Regularization-based methods are the most widely used methods, which can be divided into two subcategories: the penalty calculation method and the knowledge distillation method.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II

RESULTS (%) OF THREE CLASSES FIRST AND THEN OTHER THREE CLASSES

Method class	step 1 Urban	Agriculture	Rangeland	step 2 Forest	Water	Barren	overall mIoU
FT	0.0	0.0	0.0	63.8	72.1	47.9	30.6
PI [39]	1.2	3.0	5.4	63.5	71.9	47.6	30.5
EWC [40]	1.1	3.3	4.7	57.3	75.5	44.3	32.1
RW [63]	1.3	3.2	5.2	60.9	72.7	46.5	31.0
LWF [31]	48.1	57.2	23.3	52.3	41.5	28.3	41.7
LWF-MC [20]	67.1	69.7	34.8	50.8	49.1	38.4	51.6
ILT [30]	62.2	63.7	23.9	53.6	46.2	35.2	47.4
MiB [29]	67.6	64.0	28.6	19.7	17.5	13.7	35.1
DisALL [19]	70.1	69.1	43.0	55.3	50.7	44.4	55.4
S&R	38.2	45.0	14.7	55.6	64.6	40.5	43.1
Ours	72.2	69.0	55.5	58.1	65.2	40.8	**60.1**
Joint	73.3	80.1	56.1	62.0	76.5	64.5	68.6

"FT" represents the fine-tuning approach, and 'S&R' denotes Store and Replay method. Color blue represents the best result except our method.

TABLE III

RESULTS (%) OF TRAINING TWO CLASSES AT A STEP

Method class	step 1 Urban	Agriculture	step 2 Rangeland	Forest	step 3 Water	Barren	overall mIoU
FT	0.0	0.0	0.0	0	75.4	55.6	21.8
PI [39]	0.7	1.3	3.3	5.4	78.3	56.5	24.3
EWC [40]	0.6	1.1	2.0	5.2	77.5	54.3	23.5
RW [63]	0.7	1.2	4.2	6.9	74.6	52.5	23.4
LWF [31]	43.3	46.6	25.1	41.2	46.3	30.7	34.2
LWF-MC [20]	47.2	67.6	36.7	40.3	49.4	38.2	46.6
ILT [30]	53.2	58.9	27.7	40.6	50.0	43.4	45.6
MiB [29]	60.1	54.3	20.0	19.9	29.1	26.8	35.0
DisALL [19]	55.6	66.7	46.4	47.9	57.8	45.3	53.2
S&R	37.1	44.6	15.3	37.8	67.2	52.4	42.4
Ours	59.0	66.4	50.3	49.5	72.1	59.0	**59.4**
Joint	73.3	80.1	56.1	62.0	76.5	64.5	68.6

TABLE IV

RESULTS (%) OF TRAINING ONE CLASS AT A STEP

Method class	step 1 Urban	step 2 Agriculture	step 3 Rangeland	step 4 Forest	step 5 Water	step 6 Barren	overall mIoU
FT	0.0	0.0	0.0	0.0	0.0	86.7	14.4
PI [39]	0.1	0.1	0.4	1.1	0.3	71.7	12.4
EWC [40]	0.0	0.1	0.0	1.3	0.7	73.0	12.5
RW [35]	0.0	0.0	0.0	2.1	1.1	74.9	13.0
LWF [31]	40.2	44.4	2.0	29.4	45.7	28.5	31.7
LWF-MC [20]	32.9	56.8	11.5	39.3	72.2	44.4	42.8
ILT [30]	35.5	57.8	16.6	35.9	65.0	49.0	43.3
MiB [29]	52.1	50.4	20.2	20.4	34.7	33.9	35.3
DisALL [19]	52.4	62.1	29.9	42.2	73.1	55.1	52.4
S&R	31.4	38.0	13.1	32.2	36.1	70.2	36.8
Ours	54.7	63.5	40.7	42.6	74.2	66.4	**57.0**
Joint	73.3	80.1	56.1	62.0	76.5	64.5	68.6

a) Penalty calculation methods: For penalty calculation methods, elastic weight consolidation (EWC) [40], path integral (PI) [39], and Riemannian walks (RWs) [35] are chosen. EWC uses the empirical Fisher matrix, and PI uses the learning trajectory to compute the importance of each parameter for old classes. RW combines EWC and PI in a unique model.

b) Knowledge distillation methods: For knowledge distillation methods, Learning without Forgetting (LwF) [31], LwF MULTICLASS (LwF-MC) [20], incremental learning techniques (ILTs) [30], and DisALL [19] are chosen. LwF uses basic cross-entropy and distillation losses, and shares the same label space and classifier. LwF-MC is the single-head version of [31]. ILT [30] is the method specifically proposed for semantic segmentation, and it uses a distillation loss in the output space. MiB [29] noticed the problem of background shift in the CSS task and solved it. DisALL [19] is an improved version of ILT, with an intensive distillation of the feature section and a more advanced pseudolabel production method for the output part.

B. Results

In this section, we show and analyze the experimental results. To fully demonstrate the performance of each method, we conduct experiments in three settings: learning three classes at a step, learning two classes at a step, and learning one class at a step, which is the same with DisALL [19]. They are graded from least challenging to most challenging, i.e., in the setting where learning one class at a step is the most challenging one and suffers the greatest catastrophic forgetting. Next, we introduce and analyze the results under the three settings.

Learning three classes in a step is a simple setting, and the results are shown in Table II. It can be seen that all methods can be divided into three ranks. The first rank is FT and penalty-based methods, including PT, EWC, and RW. These methods perform poorly, and almost all of the old classes are forgotten. The second rank is the methods based on distillation, which have made great progress compared with the first rank, indicating the importance of distillation for class-incremental learning. The second rank includes LWF, LWF-MC, ILT, MiB, and DisALL. The third rank is S&R and our methods. S&R can only store and replay, and the effect is similar to LWF. Our approach outperforms all previous approaches and shows the best performance under all conditions. More importantly, we can see that the performance of all classes is fairly balanced, For example, class rangeland with bad performance previously shows the most significant improvement. Under the setting that learning three classes at a step, our method achieves a 4.7% gain compared with the previous SOTA method DisALL. Learning two classes in a step is slightly more difficult, and the results are shown in Table III. The overall result is similar to learning three classes at once. In this setting, our method is improved by 6.2% over the previous best method. Under the setting that learning one class at a step, our method is still the best and 4.6% higher than the previous best method DisALL, as shown in Table IV.

The FT method works well on class-incremental segmentation tasks for natural images. However, it has a poor performance in aerial images, especially for some natural classes, such as vegetation and forest, which are worse than some classes, such as building [18]. Classes such as vegetation have no fixed shape and rely mainly on low-level features, such as texture and color. Small changes in these features will have a big impact on the classifier. However, the classes such as building and classes on natural images have strong high-level semantic features, and the small changes do not cause more interference. In a nutshell, it is the different classes that make the different performances of the FT method between natural

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

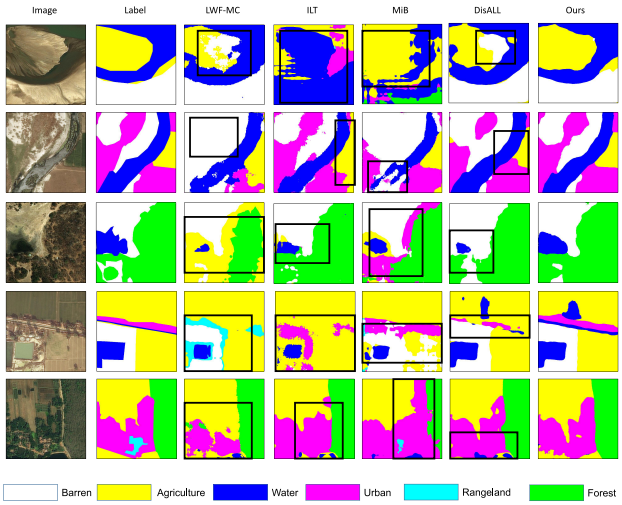


Fig. 4. Visualization of the results of our method and other SOTA methods. The black box indicates the error area. Cyan for urban, yellow for agriculture, purple for rangeland, green for forest, blue for water, white for barren land, and black for the background.

and aerial images. Due to the characteristics of aerial images, the performance of the FT method is poor.

Under all three settings, our approach achieves more than 13% gains over the baseline (ILT). Combining these three situations, it can be seen that LWF-MC has a better effect on semantic segmentation than LWF. MiB has a very good effect on memorizing old classes but is poor at learning new classes. The performance of the S&R method is basically the same as that of LWF, but the performance of learning new classes is better. The learning of new classes and keeping the memory of old classes are balanced for ILT and DisALL, but there is still a big gap in the overall performance compared with our method. In a nutshell, our approach is much better than the previous approaches, and improvements are more significant for the classes with poor performances.

1) Visualization and Analysis: The result of the visualization is shown in Fig. 4, which shows that our method is visually better than other methods. Compared with the previous methods, our method has obvious improvement in the integrity of segmentation, the accuracy of boundary regions, and the accuracy of confusable classes.

2) Limitation and Future Works: It can be seen from Fig. 4 that our method has limitations in the segmentation accuracy of small targets. In the fifth line of the figure, the network completely ignores the urban area in the middle and lower half. In the fourth line of the figure, part of agriculture is wrongly classified as water. Classes and regions that are prone to errors in segmentation tasks, such as small objects or confusing categories, are most likely to be forgotten in incremental learning. Thus, for these particularly difficult examples, how to keep their memory is a breakthrough point in the future. Keeping a small number of difficult examples is a good solution, and how to select the difficult examples is the most critical problem to be solved.

3) Visualization of Intermediate Results: The results of each step visualization are shown in Fig. 5. It can be seen that FT, PI, and RW all perform poorly. These methods are perfect

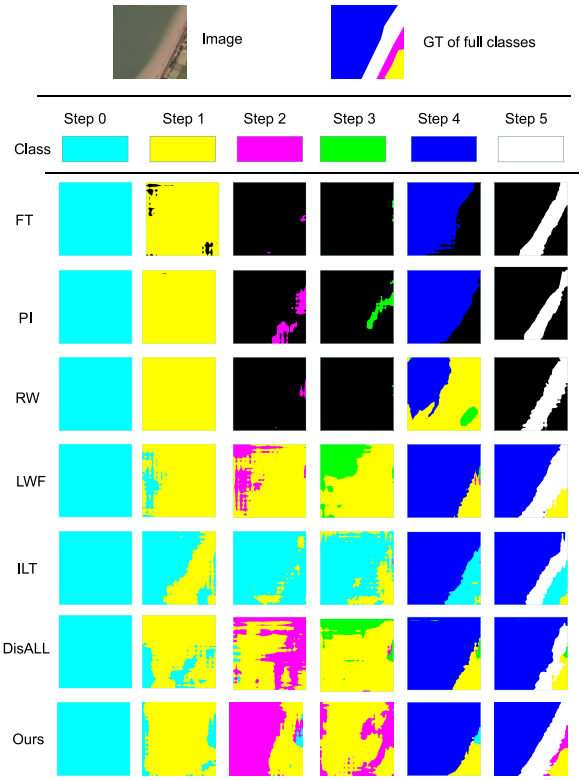


Fig. 5. Visualization of intermediate results for each step. Black represents the class background.

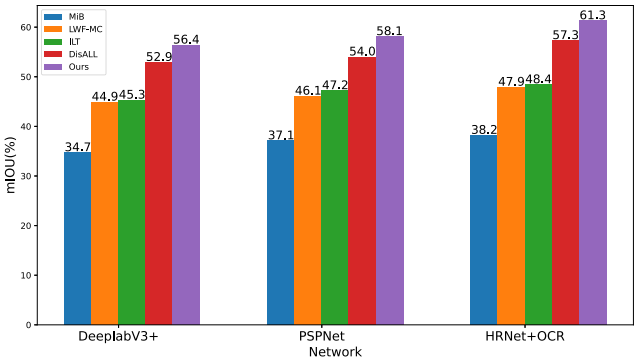


Fig. 6. Results based on different segmentation networks.

to learn new classes but completely forgetting the old class, resulting in only the last class being kept. LWF, ILT, DisALL, and ours achieve significantly improved results. However, other works have poor results for classes with small sample sizes, such as rangeland (step 2), while our work can segment it, which demonstrates the effectiveness of our method.

4) Results on Different Segmentation Networks: All of our previous experiments have been conducted on Deeplabv3, and in fact, the segmentation network can be any existing network. In order to verify the validity and universality of our method, experiments are carried out on PSPNet [3] and DANet [2]. The setup that we use is to learn one class at a step. The results of the experiment are shown in Fig. 6, and *halfVAE* is after the pyramid pooling module (PPM) in PSPNet and

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPUTATIONAL COST OF EACH MODULE

Model	Module	Paramters	FLOPs
Our Work	The PFG Module	5.4MB (5.3%)	22.7GFLOPs (5.2%)
	Segmentation Network	96.4MB (94.7%)	410.1GFLOPs (94.8%)
Other Works		96.4MB	410.1GFLOPs

after the attention operations in DANet. To ensure fairness, all settings are the same except the segmentation network. It can be seen from the results that our method is also better than the previous methods on PSPNet and DANet, which demonstrates the robustness of our method for different segmented networks. We also use HRNet + OCR [64] as the basic segmentation network. Different from PSPNet and DANet, the backbone used by HRNet + OCR is HRNet instead of ResNet, so we change the last layer of the original ResNet to the last layer of HRNet for intermediate feature distillation. The specific backbone is HRNet-W48. The experimental results are shown in Fig. 6. The effect of HRNet + OCR is significantly better than that of PSPNet and DANet. Most importantly, whether on PSPNet, DANet, or HRNet + OCR, our method is better than the previous incremental learning methods.

5) Computation Cost Analysis: We count the floating point operations (FLOPs) and parameters of the PFG module and the segmentation network, and the results are shown in Table V. Parameters represent the size of the model, with the proportion of the entire framework in parentheses. FLOPs represent the computational cost. Since our method and other methods do not change the structure of the segmentation network, the parameters and computational cost of other methods are the same as the segmentation network in the table. As can be seen, compared with the computational cost of the segmentation network, the increased computational cost of the PFG module is very small.

C. Results on Other Datasets

1) iSAID: The iSAID dataset [65] is captured by Gao Fen (GF)-1 and GF-2 satellites and contains 2806 RS images. It consists of 1411 images in the training set, 458 images in the validation set, and 957 images in the test set. The classes and proportions are shown in Table VI. Since the labels in the test set are not available, 20% of the training set is used for validation, and the validation set is used for the test, which is consistent with [17]. The training images (all resized to 512 × 512 pixels) are randomly cropped and flipped to enhance the samples.

2) Potsdam and Vaihingen: The Vaihingen dataset is composed of 33 aerial images collected of the city of Vaihingen. The average size of each image is 2494 × 2064 pixels. Following the setup in [18], we select 11 images for training, and the remaining five images (image IDs: 11, 15, 28, 30, and 34) are used to test our model. The Potsdam dataset consists of 38 high-resolution aerial images. To train and evaluate networks, we utilize ten images for training and build the test set with the remaining images (image IDs: 02 11, 02 12, 04 10, 05 11, 06 07, 07 08, and 07 10), which follows the setup in [18]. The classes and proportions of Potsdam and Vaihingen are shown in Table VII. The training images (all resized to 512 × 512 pixels) are randomly cropped and flipped to enhance the samples.

3) Results on Potsdam and Vaihingen: We conduct experiments on Potsdam and Vanhingen, and compare with SOTA methods [18] on this dataset. Potsdam and Vaihingen are more balanced than Deepglobe, as shown in Table VII. The results are shown in Tables VIII and IX. 4-1 means that the network is trained with four classes first and then trained with one. The others (3-2, 2-3, and 2-2-1) are the same. Joint (called static learning in [18]) means that all classes are accessible at one time. Similar to the results on the Deepglobe dataset, both FT and EWC perform poorly. TANet [18] is superior to DisALL in Vanhingen (4-1, 3-2) due to its enhancement of edge areas and small targets. However, DisALL is better than [18] when the number of tasks increases. The performance of various methods is similar to joint learning on Potsdam and Vanhingen, and we analyze this phenomenon from the characteristics of the datasets. Objects in Potsdam and Vaihingen are clustered, and there are multiple classes in one patch (including many old classes), which makes the network easily acquire good memory of old classes through distillation loss when learning new classes. Due to the addition of feature generation, the performance of our method is improved in each learning condition.

4) Results on iSAID: In order to compare with HIGNet [17] and show the results of our proposed method on a more challenging dataset, we conduct experiments on iSAID. ISAID is a highly unbalanced dataset with 15 classes, in which the proportion of class background is more than 97%, as shown in Table VI, which makes various methods show different performances on this dataset. The results of the experiment are shown in Table X, where 14-1 means learning 14 classes and then one class. 10-5 is learning ten classes and then learning five classes. 5s is learning five classes in one step. 10-1s is learning ten classes first and then learning one class at a step. FT, EWC, PI, and RW perform poorly as in Deepglobe. MiB performs very well in iSAID, and iSAID is with a large amount of class background, which demonstrates the function of MiB on background shift. HIGNet performs very well in this very challenging dataset. Since we add the module of PFG and TKD, there are certain improvements in all the learning settings compared with SOTA methods.

D. Results Combined With Other Class-Incremental Learning Methods

Our approach is based on the generative method and keeps the distillation at the output level. We will demonstrate through experiments that our method can be perfectly combined with other methods. Since learning one class at a time is the most challenging and most suitable for practical application scenarios, we conduct experiments and analyses in this setting. The experimental results are shown in Table XI; it can be seen that our method can be combined with the existing methods, and the performance of the existing method can be greatly improved. Even the previous best method DisALL can achieve a significant improvement.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PROPORTION FOR EACH CLASS ON iSAID

Class	BG	SH	ST	BD	TC	BC	GTF	BR	LV	SV	HC	SP	RA	SBF	PL	HA
Proportion	97.1	0.35	0.12	0.06	0.27	0.08	0.20	0.04	0.33	0.38	0.01	0.03	0.04	0.40	0.25	0.28

iSAID includes 15 foreground categories, i.e., plane (PL), baseball diamond (BD), bridge (BR), ground track field (GTF), small vehicle (SV), large vehicle (LV), ship (SH), tennis court (TC), baseball court (BC), storage tank (ST), soccer ball field (SBF), roundabout (RA), harbor (HA), swimming pool (SP), helicopter (HC), and background (BG).

TABLE VII
PROPORTION FOR EACH CLASS ON DEEPGLOBE, VAIHINGEN, AND POTSDAM

Deepglobe	Class	Urban	Agriculture	Rangeland	Forest	Water	Barren
	Proportion	9.3	57.7	10.2	13.8	3.7	6.1
Vaihingen	Class	Impervious Surfaces	Buildings	Low Vegetation	Trees	Cars	Cluster
	Proportion	30.8	28.8	20.3	13.7	1.8	4.5
Potsdam	Class	Impervious Surfaces	Buildings	Low Vegetation	Trees	Cars	Cluster
	Proportion	29.0	26.7	20.0	22.0	1.2	0.9

Blue represents the proportion of the largest class and red represents the proportion of the smallest class.

TABLE VIII
RESULTS (%) ON POTSDAM

Method	4-1 (2 steps)		3-2 (2 steps)		2-3 (2 steps)	
	mF1	mIoU	mF1	mIoU	mF1	mIoU
FT	73.6	59.3	82.7	70.6	83.6	72.1
EWC [40]	78.9	65.5	83.5	75.2	84.7	72.2
ILSS	76.8	65.5	77.5	65.8	75.2	63.2
MiB [29]	81.5	69.1	82.5	70.6	83.8	72.2
ILT [30]	80.9	68.4	81.4	69.2	82.1	70.4
TANet [18]	86.7	76.7	84.1	76.0	85.7	75.1
DisALL [19]	87.0	**77.6**	85.1	77.2	86.3	76.7
Ours	**87.3**	77.5	**86.7**	**77.5**	**86.9**	**77.7**
Joint	87.3	77.8	87.3	77.8	87.3	77.8

TABLE IX
RESULTS (%) ON VAIHINGEN

Method	4-1 (2 steps)		3-2 (2 steps)		2-3 (2 steps)		2-2-1 (3 steps)	
	mF1	mIoU	mF1	mIoU	mF1	mIoU	mF1	mIoU
FT	25.4	17.6	68.8	57.0	66.8	55.2	16.6	10.5
EWC [40]	29.9	21.1	74.6	59.8	78.8	65.0	65.3	53.9
ILSS	66.0	55.3	67.2	56.5	76.7	65.3	65.7	54.3
MiB [29]	84.2	74.0	84.5	73.2	84.2	73.7	83.7	72.8
ILT [30]	84.0	72.7	83.7	72.3	83.4	73.5	83.2	72.4
TANet [18]	86.0	**75.7**	85.3	74.8	84.7	74.4	84.7	74.0
DisALL [19]	85.4	74.9	85.0	74.6	84.9	74.5	84.7	74.1
Ours	**86.5**	75.0	**86.0**	**74.9**	**85.7**	**74.7**	**85.6**	**74.6**
Joint	86.7	75.1	86.7	75.1	86.7	75.1	86.7	75.1

This phenomenon is also easy to understand. The segmentation network is mainly composed of two parts: the feature extraction part and the classifier. The previous feature distillation methods are to keep the memory of old classes for the feature extraction part. The distillation of the output part can also guarantee the memory for the old classes of the classifier. Our approach also improves on these two parts, except that we keep the memory of the old class by generating features of the old class. Therefore, these methods have the same optimization objective as the previous method, and the proposed modules are not repeated or conflicted, which makes the combination of the two methods can obtain better results.

E. Ablation Study

We propose the PFG module and the TKD module. In this section, we verify the respective roles of the two. Besides, we compare them with other similar methods. First, we verify the respective roles of the PFG module and the TKD module. Then, we introduce the PFG module and the comparison of image generation and image-level feature generation (IFG). Finally, the effects of TKD, GKD, and CKD are shown.

1) Respective Roles of the PFG Module and the TKD Module: We propose the PFG module and the TKD module, so it is necessary to verify their respective roles. We conduct experiments in the most challenging setting, i.e., learning one class at a step. The experimental results are shown in Table XII. The setting without the PFG module and the TKD module is actually the result of baseline (ILT). We can see that the PFG achieves 11.9% gains, while the TKD module achieves 7% gains, and both are all very significant improvements. A combination of the two yields the best results. In conclusion, both the PFG module and the TKD module have very significant effects, and the PFG module is more obvious. The best results can be obtained by combining the two.

2) Pixel-Level Feature Generation Versus Image-Level Feature Generation Versus Image Generation: There are three ways to generate samples of old classes: image generation, IFG, and PFG. In this section, we show the performance of the three approaches, and the results are shown in Table XIII. The baseline represents the result of the combination of ILT and the TKD module without using any generation method. The network used for image generation is BigGAN-deep [66], which has shown stable and efficient performance in image generation. It can be seen that image generation makes the performance worse than the baseline. This phenomenon can

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

TABLE X
RESULTS (%) ON iSAID

Method	14-1 (2 steps)			10-5 (2 steps)			5s (3 steps)				10-1s (6 steps)		
	1-14	15	all	1-10	10-15	all	1-5	5-10	11-15	all	1-10	11-15	all
FT	0.32	**84.67**	5.94	0.00	**51.10**	17.03	0.00	0.00	**47.64**	15.88	0.00	15.63	5.21
PI [39]	0.28	84.47	5.89	0.00	50.64	16.88	0.00	0.00	47.61	15.87	0.00	15.59	5.20
EWC [40]	3.58	71.99	8.14	1.31	36.10	12.90	0.01	2.00	22.27	8.09	0.00	12.01	4.00
RW [63]	2.27	76.41	7.21	0.46	38.80	13.27	0.00	1.89	23.78	8.56	0.00	11.44	3.81
LWF [31]	39.57	67.84	41.46	44.70	45.41	44.94	36.53	**26.70**	38.44	33.89	4.99	16.81	8.93
LWF-MC [20]	41.98	39.54	41.81	50.96	24.11	42.01	42.08	5.20	23.00	23.42	12.19	6.09	10.16
ILT [30]	41.00	59.67	42.24	49.25	35.14	44.55	40.80	9.68	31.30	27.26	15.58	19.46	16.87
MiB [29]	48.38	81.96	50.62	47.80	47.34	47.65	**46.75**	17.83	37.54	34.04	25.06	22.94	24.35
HIGNet	48.06	70.40	49.55	52.52	39.52	48.19	43.54	23.94	35.67	34.39	31.48	**27.65**	30.21
Ours	**50.21**	70.23	**51.54**	**53.81**	40.22	**49.28**	44.60	25.89	34.73	**35.07**	**34.35**	26.43	**31.71**
Joint	56.58	85.37	58.50	58.81	57.88	58.50	57.63	59.98	57.88	58.50	58.81	57.88	58.50

"FT" represents the fine-tuning approach

TABLE XI
RESULTS COMBINED WITH OTHER METHODS

Method	mIoU (%)
MiB [29]	35.3
MiB [29] + Ours	49.0 (+13.7)
DisALL [19]	52.4
DisALL [19] + Ours	59.2 (+6.8)

TABLE XII
RESPECTIVE ROLES OF THE PFG MODULE AND THE TKD MODULE

PFG Module	TKD Module	mIoU (%)
		43.3
✓		55.2
	✓	50.3
✓	✓	57.0

TABLE XIII
PERFORMANCE OF IMAGE AND FEATURE GENERATION

Method	mIoU (%)
Baseline	50.3
Image	48.3
Imgae-level Feature	53.1
Pixel-level Feature (Ours)	57.0

be explained. When employing image generation, the corresponding pixel-level labels of the image cannot be obtained, so we use the model trained in the previous steps to segment the image and take the result as a pseudolabel. This way will inevitably produce pseudolabels with errors, and image generation is very difficult. Therefore, image generation does not bring improvement but debasement.

BigGAN-deep [66] is also used to generate image-level features. The difference is that we get the intermediate features instead of images. We set that all the features of the image belong to one class. It can be seen from the table that image feature generation is much better than image generation. This

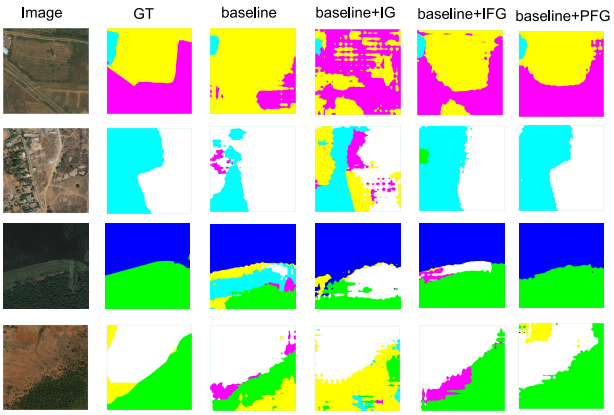


Fig. 7. Comparison of PFG and other generation methods. IG stands for image generation, and IFG is image-level feature generation.

phenomenon is easy to understand. The dimension of features is much higher than the image, so it contains more information and, thus, is classifier friendly.

Our PFG method achieves the best result because *halfVAE* and multiple operations guarantee the consistency and diversity of the features, and also, the specially designed loss can guarantee the class-incremental learning of the PFG module. In general, our method takes into account the characteristics of both semantic segmentation and class-incremental learning, so it achieves the best performance.

The visualization of various generation methods is shown in Fig. 7. The baseline is to remove the PFG module from the full-version network. It can be seen that both IFG and PFG are better than the baseline. However, IG is worse than the baseline, which demonstrates that generating images is much more challenging than generating features, and so the generated results are poor. PFG outperforms IFG because the pixel-level pattern is more consistent with the segmentation task.

The visualization results of the features generated by PFG and extracted features are shown in Fig. 8. Compared to the extracted features, the generated features are more clustered and more responsive. The feature extraction network can keep

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

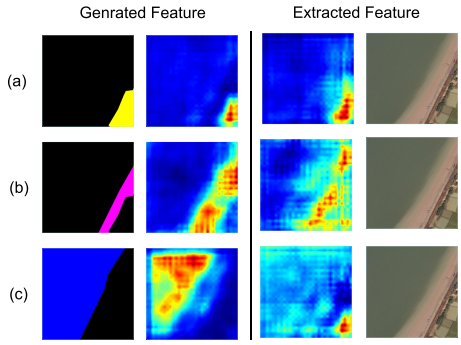


Fig. 8. Visualization results of generated features and extracted features at learning steps. (Left) Pseudolabel and the features generated by PFG. (Right) Extracted features and images. (a) After learning agriculture. (b) After learning agriculture and rangeland. (c) After learning agriculture, rangeland, and water.

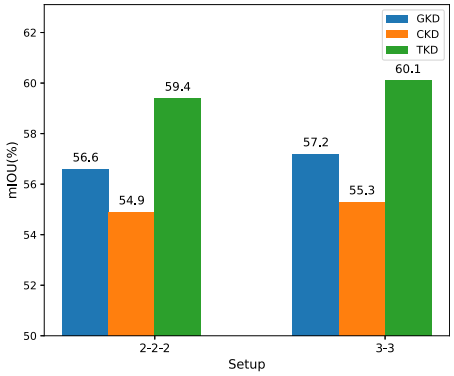


Fig. 9. Results under different distillation methods. 2-2-2 denotes learning two classes at a step, and 3-3 denotes learning three classes at a step.

the response to the features of the old class when learning the new class. Correspondingly, the extracted features should be relatively scattered. Corresponding to the pseudolabel, the features generated by PFG are accurate during the learning process.

3) TKD Versus Other Knowledge Distillation Methods: The corresponding results of different distillation methods are shown in Fig. 9, where 2-2-2 represents learning two classes at a step and 3-3 represents learning three classes at a step. Except for the different distillation methods, the remaining modules, such as PFG, remain consistent. The reason for not using learning one class at a step as the setting is that TKD and CKD are the same in this condition. As can be seen from the table, CKD has the worst effect because it does not compare classes and uses sigmoid to normalize directly, resulting in a small value in logits for all classes. GKD compares all classes together, causing the output of the new class to squeeze the old class. Our approach is to compare (i.e., calculate softmax) classes at the same learning step, and classes in different learning steps do not interfere with each other. Therefore, TKD avoids the defects of GKD and CKD and, thus, achieves the best result.

F. Ratio of Old and New Features

We determine the proportion of each old class by taking the inverse of the IoU of each class. The key to class-incremental

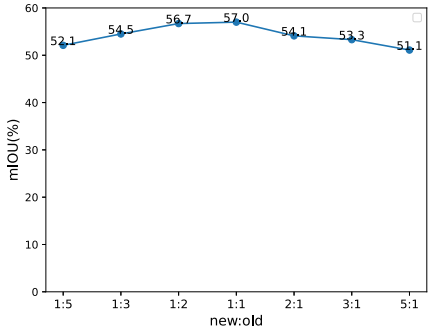


Fig. 10. Results under different ratios of old and new features.

TABLE XIV
RESULTS (%) WITH DIFFERENT WEIGHT VALUES

	λ_1	λ_2	λ_3	λ_4	λ_5
10	56.8	56.9	56.5	56.9	57.0
100	57.0	57.0	57.0	57.0	57.1
200	57.0	57.0	57.1	56.6	56.9

learning is to keep the memory of the old class while learning the new classes, so the ratio of the new class to the old class is very important. In this section, we introduce the results of different proportions of old and new features. The old and new classes ratio is obtained by counting the true labels (for new classes) and generated labels (for old classes). Specifically, when generating the features of the old classes, we need pseudolabels of the old classes as the input, and we can control the ratio of the old and new classes through the pseudolabel i.e., $\tilde{\mathbf{Y}}$ in Fig. 1.

We use learning one class at a step as the setting. As can be seen from the experimental results in Fig. 10, when the ratio of old to new is 1:1, the result is the best, which is consistent with our previous expectations. In contrast, the drop is greater when the number of generated old-class features drops, which highlights the importance of reviewing old class features and also demonstrates the significance of generating features of old classes.

G. Results With Different Weight Values

This section verifies the influence of different weight values in (11) on the final performance. The results of the experiments with different weights of losses are shown in Table XIV, and the table should be analyzed column by column. The experiments are conducted on the Deepglobe dataset [4], and the segmentation network is Deeplabv3 [55] with ResNet-101 [60] as the backbone. It should be noted that, when a specific weight changes, the rest of the weights are set to default and remain fixed. It can be seen that, since there are multiple ways to keep the memory of the old class and learn the new class, the effect of a certain weight change on the final result is not apparent. Next, we analyze the five weights one by one.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

TABLE XV
RESULTS (%) OF DIFFERENT LEARNING ORDERS

Method	step 1	step 2	step 3	step 4	step 5	step 6	mIoU	Average
ILT	Agriculture	Barren	Water	Forest	Rangeland	Urban	45.1	42.6±2.2
	Urban	Agriculture	Rangeland	Forest	Water	Barren	43.3	
	Barren	Water	Forest	Rangeland	Agriculture	Urban	39.1	
	Forest	Urban	Barren	Agriculture	Water	Rangeland	42.7	
LWF-MC	Agriculture	Barren	Water	Forest	Rangeland	Urban	44.7	42.0±2.4
	Urban	Agriculture	Rangeland	Forest	Water	Barren	42.8	
	Barren	Water	Forest	Rangeland	Agriculture	Urban	38.2	
	Forest	Urban	Barren	Agriculture	Water	Rangeland	42.1	
DisALL	Agriculture	Barren	Water	Forest	Rangeland	Urban	55.9	52.5±2.7
	Urban	Agriculture	Rangeland	Forest	Water	Barren	52.4	
	Barren	Water	Forest	Rangeland	Agriculture	Urban	48.4	
	Forest	Urban	Barren	Agriculture	Water	Rangeland	53.1	
Ours	Agriculture	Barren	Water	Forest	Rangeland	Urban	58.3	**56.5**±1.5
	Urban	Agriculture	Rangeland	Forest	Water	Barren	57.0	
	Barren	Water	Forest	Rangeland	Agriculture	Urban	54.1	
	Forest	Urban	Barren	Agriculture	Water	Rangeland	56.6	
Joint								

λ_1 is the coefficient of $\mathcal{L}_{\text{dis}}$, and $\mathcal{L}_{\text{dis}}$ represents the knowledge distillation at the feature level of the segmentation network. The weight value of 100 and 200 can achieve the same performance. When the value is 10, the result will drop slightly, which indicates that the feature level distillation plays an important role. λ_2 is the weight of $\tilde{\mathcal{L}}_{\text{dis}}$, which represents the knowledge distillation of the PFG module, and the regularity of λ_2 is similar to that of λ_1. λ_3 represents the weight of the task-wise distillation loss $\mathcal{L}_{\text{tkd}}$. If λ_3 is set too small, the classifier will forget the old class quickly, so the effect will decrease, as shown in the performance when the λ_3 is 10. Meanwhile, the value of 200 is better than 100 (default), which means that, in remote sensing datasets, such as Deepglobe, $\mathcal{L}_{\text{tkd}}$ with large weights can get better performance. λ_4 represents the weight of reconstruction loss $\mathcal{L}_{\text{rec}}$. The purpose of $\mathcal{L}_{\text{rec}}$ is to make the generator learn new classes. It can be seen that, when λ_4 is large (like 200), the effect will be reduced. It is easy to understand. When λ_4 is large, the feature generation module learns new classes excessively, and the forgetting of old classes will be accelerated, thus losing the effect of replaying features of old classes. A good way is to balance the memory and learning of the feature generation module, i.e., $\lambda_2 = \lambda_4$. λ_5 represents the weight of $\mathcal{L}_{\text{kl}}$, and $\mathcal{L}_{\text{kl}}$ is to make the extracted features satisfy a specific distribution. It can be seen that the value of λ_5 has little effect on the final result in the range of 10–200.

H. Influence of Different Learning Orders

The order of learning categories has a significant effect on the final result, which is also discussed in DisALL [19]. Intuitively, networks are more likely to get better results when classes that make up a large part of the dataset appear in early learning because pixels that belong to these big classes will also appear in later learning and, thus, can be reviewed frequently. However, when applied in the real world, it is uncertain which class comes first and what proportion of it is. Therefore, in order to fully demonstrate the performance of the network, we conduct experiments on different training sequences. The result is shown in Table XV. It can be seen that different training sequences have a great impact on the final results. Aerial images are imbalanced, which leads to the fact that some classes may never be encountered after the current learning, as discussed in Section I. However, the proposed PFG module can generate any number of features of old classes as required so that different learning sequences have the least influence on the final results (i.e., the results are with the least fluctuation). This demonstrates the great value of our generative approach in the practical use of the real world.

V. CONCLUSION

In this article, to solve the catastrophic forgetting problem, targeted methods are proposed in both generative and distillation aspects. The two methods first take into account and easily solve the problem of class imbalance, including the imbalance between new classes and old classes, and the imbalance between old classes. Meanwhile, the commonly used image generation method is abandoned, and the PFG method is adopted. The results show that the effect of feature generation is outstanding compared with image or IFG. The success of PFG demonstrates that the generative method can also play an important role in class-incremental learning of semantic segmentation. How to generate more diverse and realistic features, and designing the distillation method that conforms to the class-incremental learning task, maybe the direction for future studies.

REFERENCES

[1] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder–decoder with atrous separable convolution for semantic image segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 801–818.

[2] J. Fu et al., "Dual attention network for scene segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2019, pp. 3146–3154.

[3] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2881–2890.

[4] I. Demir et al., "DeepGlobe 2018: A challenge to parse the Earth through satellite images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2018, pp. 172–181.

[5] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. (2012). *The PASCAL Visual Object Classes Challenge 2012 (VOC2012) Results*. [Online]. Available: http://www.pascal-network.org/challenges/VOC/voc2012/workshop/index.html

[6] Q. Ming, L. Miao, Z. Zhou, and Y. Dong, "CFC-Net: A critical feature capturing network for arbitrary-oriented object detection in remote-sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–14, 2022.

[7] L. Ding et al., "Looking outside the window: Wide-context transformer for the semantic segmentation of high-resolution remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2022.

[8] X. Zheng, X. Chen, X. Lu, and B. Sun, "Unsupervised change detection by cross-resolution difference learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–16, 2022.

[9] Z. Li et al., "Cascaded multiscale structure with self-smoothing atrous convolution for semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2021.

[10] L. Ding, H. Tang, Y. Liu, Y. Shi, X. X. Zhu, and L. Bruzzone, "Adversarial shape learning for building extraction in VHR remote sensing images," *IEEE Trans. Image Process.*, vol. 31, pp. 678–690, 2022.

[11] H. Zhang et al., "ResNeSt: Split-attention networks," 2020, *arXiv:2004.08955*.

[12] F. Ozdemir, P. Fuernstahl, and O. Goksel, "Learn the new, keep the old: Extending pretrained models with new anatomy and images," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.* Cham, Switzerland: Springer, 2018, pp. 361–369.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

[13] Y. Feng, X. Sun, W. Diao, J. Li, X. Gao, and K. Fu, "Continual learning with structured inheritance for semantic segmentation in aerial imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–17, 2022.

[14] R. M. French, "Catastrophic forgetting in connectionist networks," *Trends Cogn. Sci.*, vol. 3, pp. 128–135, May 1999.

[15] I. J. Goodfellow, M. Mirza, D. Xiao, A. Courville, and Y. Bengio, "An empirical investigation of catastrophic forgetting in gradient-based neural networks," 2013, *arXiv:1312.6211*.

[16] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.

[17] X. Rong, X. Sun, W. Diao, P. Wang, Z. Yuan, and H. Wang, "Historical information-guided class-incremental semantic segmentation in remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–18, 2022.

[18] J. Li et al., "Class-incremental learning network for small objects enhancing of semantic segmentation in aerial imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–20, 2021.

[19] L. Shan, W. Wang, K. Lv, and B. Luo, "Class-incremental learning for semantic segmentation in aerial imagery via distillation in all aspects," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–12, 2022.

[20] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "ICaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2001–2010.

[21] F. M. Castro, M. J. Marín-Jiménez, N. Guil, C. Schmid, and K. Alahari, "End-to-end incremental learning," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 233–248.

[22] Y. Wu et al., "Incremental classifier learning with generative adversarial networks," 2018, *arXiv:1802.00853*.

[23] O. Ostapenko, M. Puscas, T. Klein, P. Jahnichen, and M. Nabi, "Learning to remember: A synaptic plasticity driven framework for continual learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 11321–11329.

[24] Z. Gu, S. Zhou, L. Niu, Z. Zhao, and L. Zhang, "Context-aware feature generation for zero-shot semantic segmentation," in *Proc. 28th ACM Int. Conf. Multimedia*, Oct. 2020, pp. 1921–1929.

[25] Z. Gu, S. Zhou, L. Niu, Z. Zhao, and L. Zhang, "From pixel to patch: Synthesize context-aware features for zero-shot semantic segmentation," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Feb. 10, 2022, doi: 10.1109/TNNLS.2022.3145962.

[26] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," 2013, *arXiv:1301.3781*.

[27] C. Doersch, "Tutorial on variational autoencoders," 2016, *arXiv:1606.05908*.

[28] M. Mirza and S. Osindero, "Conditional generative adversarial nets," 2014, *arXiv:1411.1784*.

[29] F. Cermelli, M. Mancini, S. Rota Bulo, E. Ricci, and B. Caputo, "Modeling the background for incremental learning in semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9233–9242.

[30] U. Michieli and P. Zanuttigh, "Incremental learning techniques for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop (ICCVW)*, Oct. 2019, pp. 1–8.

[31] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, Dec. 2018.

[32] K. Yuan, W. Xu, W. Li, and W. Ding, "An incremental learning mechanism for object classification based on progressive fuzzy three-way concept," *Inf. Sci.*, vol. 584, pp. 127–147, Jan. 2022.

[33] M. De Lange et al., "Continual learning: A comparative study on how to defy forgetting in classification tasks," 2019, *arXiv:1909.08383*.

[34] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter, "Continual lifelong learning with neural networks: A review," *Neural Netw.*, vol. 113, pp. 54–71, May 2019.

[35] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 532–547.

[36] C. Fernando et al., "PathNet: Evolution channels gradient descent in super neural networks," 2017, *arXiv:1701.08734*.

[37] B. Pfulb and A. Gepperth, "Overcoming catastrophic forgetting with Gaussian mixture replay," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 4548–4557.

[38] T. Xiao, J. Zhang, K. Yang, Y. Peng, and Z. Zhang, "Error-driven incremental learning in deep convolutional neural network for large-scale image classification," in *Proc. 22nd ACM Int. Conf. Multimedia*, Nov. 2014, pp. 177–186.

[39] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3987–3995.

[40] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.

[41] X. Yao, J. Han, G. Cheng, X. Qian, and L. Guo, "Semantic annotation of high-resolution satellite images via weakly supervised learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 6, pp. 3660–3671, Jun. 2016.

[42] F. Ozdemir and O. Goksel, "Extending pretrained segmentation networks with additional anatomical structures," *Int. J. Comput. Assist. Radiol. Surg.*, vol. 14, no. 7, pp. 1187–1195, Jul. 2019.

[43] O. Tasar, Y. Tarabalka, and P. Alliez, "Incremental learning for semantic segmentation of large-scale remote sensing data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 9, pp. 3524–3537, Sep. 2019.

[44] U. Michieli and P. Zanuttigh, "Knowledge distillation for incremental learning in semantic segmentation," *Comput. Vis. Image Understand.*, vol. 205, Apr. 2021, Art. no. 103167.

[45] L. Yu, X. Liu, and J. van de Weijer, "Self-training for class-incremental semantic segmentation," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Mar. 17, 2022, doi: 10.1109/TNNLS.2022.3155746.

[46] H. Huang, C. Wang, P. S. Yu, and C.-D. Wang, "Generative dual adversarial network for generalized zero-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 801–810.

[47] S. Rahman, S. Khan, N. Barnes, and F. S. Khan, "Any-shot object detection," in *Proc. Asian Conf. Comput. Vis.*, 2020, pp. 1–17.

[48] A. Cheraghian, S. Rahman, P. Fang, S. K. Roy, L. Petersson, and M. Harandi, "Semantic-aware knowledge distillation for few-shot class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 2534–2543.

[49] C. Zhu, F. Chen, U. Ahmed, Z. Shen, and M. Savvides, "Semantic relation reasoning for shot-stable few-shot object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 8782–8791.

[50] A. Cheraghian et al., "Synthesized feature based few-shot class-incremental learning on a mixture of subspaces," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Oct. 2021, pp. 8661–8670.

[51] Y. Xian, T. Lorenz, B. Schiele, and Z. Akata, "Feature generating networks for zero-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 5542–5551.

[52] E. Schonfeld, S. Ebrahimi, S. Sinha, T. Darrell, and Z. Akata, "Generalized zero- and few-shot learning via aligned variational autoencoders," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8247–8255.

[53] Y. Xian, S. Sharma, B. Schiele, and Z. Akata, "f-VAEGAN-D2: A feature generating framework for any-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 10275–10284.

[54] X. Liu et al., "Generative feature replay for class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2020, pp. 226–227.

[55] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," 2017, *arXiv:1706.05587*.

[56] W. Xian et al., "TextureGAN: Controlling deep image synthesis with texture patches," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8456–8465.

[57] J.-Y. Zhu et al., "Multimodal image-to-image translation by enforcing bi-cycle consistency," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 465–476.

[58] H. Ahn, J. Kwak, S. Lim, H. Bang, H. Kim, and T. Moon, "SS-IL: Separated softmax for incremental learning," 2020, *arXiv:2003.13947*.

[59] W. Chen, Z. Jiang, Z. Wang, K. Cui, and X. Qian, "Collaborative global-local networks for memory-efficient segmentation of ultra-high resolution images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8924–8933.

[60] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[61] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.

[62] S. R. Bulo, L. Porzi, and P. Kontschieder, "In-place activated BatchNorm for memory-optimized training of DNNs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 5639–5647.

[63] P. Dhar, R. V. Singh, K.-C. Peng, Z. Wu, and R. Chellappa, "Learning without memorizing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 5138–5146.

[64] Y. Yuan, X. Chen, X. Chen, and J. Wang, "Segmentation transformer: Object-contextual representations for semantic segmentation," 2019, *arXiv:1909.11065*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.

[65] S. W. Zamir et al., "iSAID: A large-scale dataset for instance segmentation in aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2019, pp. 28–37.
[66] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," 2018, *arXiv:1809.11096*.



Lianlei Shan (Graduate Student Member, IEEE) received the B.E. degree from the Shandong University of Science and Technology, Qingdao, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Computer and Control Engineering, University of Chinese Academy of Sciences, Beijing, China.

His research interests include remote sensing image processing and computer vision.



Weiqiang Wang (Member, IEEE) received the B.E. and M.E. degrees in computer science from Harbin Engineering University, Harbin, China, in 1995 and 1998, respectively, and the Ph.D. degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences (CAS), Beijing, China, in 2001.

He is currently a Professor with the School of Computer and Controlling Engineering, University of CAS, Beijing. His research interests include multimedia content analysis and computer vision.



Ke Lv (Senior Member, IEEE) received the B.S. degree from the Department of Mathematics, Ningxia University, Yinchuan, China, in 1993, and the master's and Ph.D. degrees from the Department of Mathematics and the Department of Computer Science, Northwest University, Xi'an, China, in 1998 and 2003, respectively.

He is currently a Professor with the University of Chinese Academy of Sciences, Beijing, China. His research focuses on curve matching, 3-D image construction, and computer graphics.



Bin Luo (Senior Member, IEEE) received the BEng. degree in electronics and the MEng. degree in computer science from Anhui University, Hefei, China, in 1984 and 1991, respectively, and the Ph.D. degree in computer science from the University of York, York, U.K., in 2002.

He was a Research Associate with the University of York from 2000 to 2004.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 11:44:20 UTC from IEEE Xplore. Restrictions apply.